

REECICLA

Documento Oficial de Arquitectura y Plan de Ejecución

Plataforma SaaS de reciclaje de equipos de línea blanca

Proyecto	Kata Arquitectónico Pollo Chingón: Volviéndose ecológico (Reecicla)
Versión	1.0
Fecha	02 de octubre de 2026
Estilo arquitectónico	Microservicios
Equipo	5 personas
Duración	4 sprints de 1 semana
Estado	Pendiente de validación de supuestos (sección 13)

Este documento consolida el análisis del kata, el documento base del equipo (módulos y HU-001 a HU-022) y las correcciones detectadas en su revisión. Donde cambia algo respecto al documento base, está marcado en la sección 12.

1. Propósito y alcance

Propósito. Construir un sistema que permita a clientes cotizar, enviar, inspeccionar y cobrar por equipos usados de línea blanca (refrigeradores, lavadoras, lavaplatos, cocinas, entre otros), y que otros proveedores puedan usar como SaaS freemium.

Dentro del alcance (MVP)

- Registro e inicio de sesión de clientes, con contexto de tenant y roles.
- Catálogo de tipos, marcas y reglas de evaluación.
- Cotización inicial, aceptación o rechazo, y cotización ajustada.
- Órdenes, solicitud de caja, envío, tracking y recepción.
- Inspección con resultado (reventa o reciclaje) y registro del destino final.
- Pago al cliente con reintentos (transferencia o cheque).
- Notificaciones por cambio de estado.
- Reportes básicos del negocio.
- Plan gratuito con límites por tenant.
- Desactivación automática de tipos sin recepciones en 12 meses.

Fuera del alcance de esta versión

- Clientes móviles y de quiosco (la API los soporta, pero no se construyen; ver supuesto S3).
- Integración real con eBay, bancos o empresas de correo (se usan adaptadores simulados).
- Pasarela de pago con tarjeta y facturación de planes de pago.
- Despliegue productivo en la nube y alta disponibilidad multi-región.

2. Requisitos

2.1 Requisitos funcionales (trazados al kata)

ID	Requisito	Origen en el kata
RF-1	Cotizar un equipo usado por canal digital	"Los clientes pueden obtener una cotización"
RF-2	Solicitar caja, embalar y enviar el equipo	"Recibirán una caja por correo"
RF-3	Inspeccionar el equipo recibido y decidir reciclar o vender	"Se lo evalúa (inspecciona)"
RF-4	Cambiar la cotización si el equipo no está como se declaró	"Se tiene el derecho de cambiar la cotización"
RF-5	Pagar al cliente por transferencia o cheque	"Transferencia bancaria o cheque"
RF-6	Mantener la lista de equipos aceptados, con altas frecuentes	"Lista de equipos que cambia con frecuencia"
RF-7	Reglas propias de cotización y evaluación por tipo	"Cada tipo tiene su propio conjunto de reglas"
RF-8	Retirar tipos sin recepciones en un año	"Se lo eliminará del sistema"
RF-9	Operar como SaaS multi-tenant con modelo freemium	"Modelo freemium... modalidad SaaS"

2.2 Requisitos no funcionales

ID	Atributo	Meta del MVP
RNF-1	Rendimiento	Cotización con p95 menor a 2 s bajo carga normal
RNF-2	Escalabilidad	Servicios sin estado, escalables de forma horizontal
RNF-3	Extensibilidad	Agregar un tipo de equipo con sus reglas sin redespargar ningún servicio
RNF-4	Aislamiento	Toda consulta filtrada por tenant_id; ningún dato cruza tenants
RNF-5	Seguridad	Contraseñas con hash (argon2/bcrypt), tokens JWT, TLS, roles verificados en el Gateway y en cada servicio
RNF-6	Auditabilidad	Todo cambio de estado y de precio queda registrado con fecha y autor
RNF-7	Consistencia	Pagos idempotentes; consistencia eventual en el resto del flujo
RNF-8	Observabilidad	Logs con correlation_id que viaja entre servicios
RNF-9	Disponibilidad	Objetivo de 99.5% para el MVP (no se garantiza más)

3. Drivers arquitectónicos

1. **Variabilidad por tipo de equipo** (RF-7, RNF-3): reglas como datos versionados (JSONB), no como código.
2. **Evolución continua** (1 a 2 tipos nuevos por mes): servicios con despliegue independiente.
3. **SaaS multi-tenant** (RF-9, RNF-4): tenant_id en todo el modelo, validado por Auth.
4. **Escala impredecible** (cientos a millones de usuarios): servicios sin estado y comunicación asíncrona.
5. **Proceso de larga duración con pasos humanos** (RF-2 a RF-5): máquina de estados con un dueño del proceso.

4. Arquitectura de módulos

4.1 Principios

- Cada servicio es dueño de sus datos y de su base PostgreSQL.
- **No existen claves foráneas SQL entre bases.** Las referencias entre servicios son identificadores lógicos y la integridad se resuelve por comunicación entre servicios.
- Un servicio puede tener varias tablas si pertenecen al mismo contexto de negocio.
- El API Gateway/BFF es infraestructura, no un módulo de negocio, y no es dueño de ninguna tabla.

4.2 Mapa de módulos

#	Módulo	Responsabilidad	Base de datos	Tablas propietarias
1	Auth / Identity (incluye Tenant)	Usuarios, autenticación, roles, tenants, planes y límites	AUTH DB	tenants, users, user_roles, sessions, tenant_plans, usage_counters
2	Catalog	Tipos, marcas y reglas de evaluación	CATALOG DB	device_types, device_brands, evaluation_rules
3	Quotation	Cálculo y registro de cotizaciones, iniciales y ajustadas	QUOTATION DB	quotes, pricing_rules

#	Módulo	Responsabilidad	Base de datos	Tablas propietarias
4	Orders	Ciclo de vida de la orden, cajas, tracking, destino final	ORDERS DB	orders, order_status_history, box_requests, dispositions
5	Inspection	Inspecciones y resultados del equipo recibido	INSPECTION DB	inspections, inspection_items
6	Payment	Intentos y estados de pago al cliente	PAYMENT DB	payments
7	Notification	Avisos al cliente por eventos	Sin BD obligatoria en MVP	(registro simple opcional)
8	Reporting	Consultas y reportes	Read model simplificado	(alimentado por eventos)
–	API Gateway / BFF	Enrutamiento, validación de token, rate limiting	Ninguna	Ninguna

4.3 Datos clave por módulo

Auth / Identity

- **tenants:** id, nombre, estado.
- **users:** id, tenant_id, correo, hash de contraseña, estado.
- **user_roles:** roles CLIENT, INSPECTOR, CATALOG_ADMIN y TENANT_ADMIN.
- **tenant_plans:** plan (FREE o PREMIUM) y límite mensual de cotizaciones.
- **usage_counters:** consumo por tenant y mes.
- El token JWT lleva user_id, tenant_id y roles como claims, por lo que los demás servicios no consultan a Auth en cada llamada.

Catalog

- **device_types:** id, tenant_id, nombre, active, last_received_at. Se inactiva, nunca se borra.
- **device_brands:** id, device_type_id, nombre, active.
- **evaluation_rules:** id, device_type_id (referencia lógica interna), version, checklist en JSONB, active.

Quotation

- **quotes:** id, tenant_id, customer_id, device_type_id (lógica), marca y modelo como texto (snapshot), año, condición declarada, quote_type (INITIAL o ADJUSTED), parent_quote_id (para ajustadas, FK interna), monto, estado, vigencia.
- **pricing_rules:** id, device_type_id (lógica), version, parámetros en JSONB, active.

Orders

- **orders:** id, tenant_id, customer_id, quote_id (lógica), snapshot de marca, modelo, año y condición declarada, estado actual.
- **order_status_history:** id, order_id, estado anterior, estado nuevo, fecha, autor.
- **box_requests:** id, order_id, dirección, estado, guía de envío. Una orden puede tener varias.
- **dispositions:** id, order_id, tipo (RESALE o RECYCLE), estado, referencia (certificado de destrucción o publicación de venta).

Inspection

- **inspections:** id, order_id (lógica), inspector, fecha, resultado (RESALE o RECYCLE), observaciones.
- **inspection_items:** id, inspection_id, criterio, valor, impacto en el precio, evidencia (referencia a foto).

Payment

- **payments:** id, order_id (lógica), idempotency_key, método (TRANSFER o CHECK), monto, estado (PENDING, FAILED, COMPLETED), fecha, motivo de fallo.
- Un índice único parcial garantiza un solo COMPLETED por orden: UNIQUE (order_id) WHERE status = 'COMPLETED'.

5. Comunicación y proceso de negocio

5.1 Quién coordina el proceso

Decisión: Orders es el **dueño del proceso** (coordinador de la saga). Los demás servicios hacen su parte y publican un evento; Orders decide el siguiente paso. Notification y Reporting solo escuchan eventos (coreografía).

- **Síncrono (REST):** consultas de catálogo y cálculo de cotización, donde el cliente espera respuesta inmediata.
- **Asíncrono (broker de mensajes):** todo el flujo posterior a la aceptación.

5.2 Eventos del dominio

Evento	Publica	Consumen
QuoteAccepted	Quotation	Orders
OrderCreated	Orders	Notification, Reporting
BoxShipped	Orders	Notification
DeviceReceived	Orders	Inspection, Catalog (actualiza last_received_at), Notification, Reporting
InspectionCompleted	Inspection	Orders, Quotation
AdjustedQuoteIssued	Quotation	Orders, Notification
AdjustedQuoteAccepted	Quotation	Orders
AdjustedQuoteRejected	Quotation	Orders
PaymentCompleted	Payment	Orders, Notification, Reporting
PaymentFailed	Payment	Orders, Notification
DispositionCompleted	Orders	Notification, Reporting

5.3 Flujo principal

1. El cliente inicia sesión (Auth) y consulta el catálogo.
2. Quotation calcula la cotización inicial; el cliente la acepta (QuoteAccepted).
3. Orders crea la orden y gestiona la caja y el envío.
4. Al recibir el equipo (DeviceReceived), Inspection registra la evaluación.
5. Si la condición real coincide con la declarada, la cotización se confirma. Si no, Quotation emite una cotización ajustada.
6. El cliente acepta o rechaza el precio ajustado.
7. Si acepta, Payment procesa el pago (con reintentos si falla).
8. Orders registra el destino final (reventa o reciclaje) y cierra la orden.

5.4 Camino de rechazo del precio ajustado

Es un hueco del kata, resuelto aquí como **supuesto S2**: si el cliente rechaza el precio ajustado, la orden pasa a `ADJUSTMENT_REJECTED` y el cliente elige entre devolución (costo a su cargo) o reciclaje sin pago. Si no responde en 7 días, se aplica reciclaje sin pago.

5.5 Estados

Orden (Orders): `CREATED` → `BOX_REQUESTED` → `BOX_SHIPPED` → `IN_TRANSIT` → `RECEIVED` → `INSPECTED` → `PRICE_CONFIRMED` o `ADJUSTMENT_PENDING` → `PAYMENT_PENDING` → `PAID` → `DISPOSED` → `CLOSED`. Estados de salida: `CANCELLED` y `ADJUSTMENT_REJECTED`.

Cotización: `QUOTED` → `ACCEPTED`, `REJECTED` o `EXPIRED`. La ajustada: `ADJUSTED_PENDING` → `ADJUSTED_ACCEPTED` o `ADJUSTED_REJECTED`.

Pago: `PENDING` → `COMPLETED` o `FAILED` (un fallo permite un nuevo intento).

Las transiciones las controla el servicio dueño y se refuerzan con `CHECKS` de valores permitidos.

6. Decisiones de consistencia (se mantienen del documento base)

1. Los tipos se inactivan, no se borran; el historial se conserva.
2. Una orden puede tener varias solicitudes de caja.
3. Orders guarda un snapshot de marca, modelo, año y condición declarada.
4. Payment permite varios intentos; los fallidos se conservan; solo un `COMPLETED` por orden.
5. Quotation y Orders conservan sus datos históricos aunque el catálogo cambie.
6. La marca es texto en Quotation y Orders (snapshot), sin FK entre bases.
7. `tenant_id` es referencia lógica; la validación pertenece a Auth.
8. Reglas de evaluación en JSONB; resultados de inspección normalizados en `inspection_items`.
9. Sin claves foráneas entre bases de microservicios.

7. Supuestos (a validar con el profesor)

ID	Supuesto
S1	Los equipos grandes (refrigeradores, cocinas) se recogen con logística propia; “caja por correo” aplica a equipos pequeños. En el MVP solo se registra el pedido y el tracking.
S2	Rechazo del precio ajustado: devolución a costo del cliente o reciclaje sin pago; sin respuesta en 7 días, reciclaje sin pago.
S3	El MVP construye el cliente web. Móvil y quiosco consumirán la misma API del Gateway sin cambios de backend.
S4	La “eliminación” de un tipo es una desactivación lógica tras 12 meses sin recepciones, con aviso previo al administrador.
S5	Cada tenant tiene su propio catálogo y reglas; un tenant ve solo lo suyo.
S6	Transferencia y cheque se simulan con un adaptador bancario; el cheque se registra de forma manual.
S7	Plan gratuito: hasta 20 cotizaciones por mes por tenant. Plan premium: sin límite.
S8	Reventa y reciclaje se registran como estados y referencias; no hay integración real con marketplaces.

8. Backlog maestro

Escala de puntos: Fibonacci. **Capacidad estimada:** ~34 puntos por sprint (5 personas, 1 semana).

Prioridad: MoSCoW. Los criterios de aceptación (CA) están en formato resumido; cada uno se desglosa en tareas al planificar el sprint.

Sprint 1: Identidad, catálogo mínimo y cotización inicial (34 pts)

Objetivo: un cliente se registra, inicia sesión y obtiene una cotización.

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación
TE-01	Infra	Must	5	Infraestructura base. Gateway/BFF, esqueleto de los servicios, contenedores Docker y CI básico. • Cada servicio arranca con docker compose up. • El Gateway enruta a todos los servicios. • El CI ejecuta pruebas en cada push.
HU-001	Auth	Must	5	<i>Como administrador de la plataforma, quiero configurar tenants y roles para aislar los datos de cada proveedor.</i> • Se crea un tenant con un administrador inicial. • Existen los roles CLIENT, INSPECTOR, CATALOG_ADMIN y TENANT_ADMIN. • El JWT incluye tenant_id y roles.
HU-002	Auth	Must	3	<i>Como cliente, quiero registrarme para gestionar mis solicitudes.</i> • El correo es único por tenant. • La contraseña se guarda con hash. • Se valida el formato de los datos.
HU-003	Auth	Must	3	<i>Como cliente, quiero iniciar sesión para acceder a mis cotizaciones y órdenes.</i> • Credenciales válidas devuelven un token con expiración. • Las inválidas devuelven error sin revelar cuál dato falló. • Un usuario solo accede a los datos de su tenant.
HU-006	Catalog	Must	5	<i>Como administrador del catálogo, quiero gestionar tipos de dispositivo para definir qué equipos se aceptan. (Adelantada desde el Sprint 2.)</i> • Crear, editar e inactivar tipos. • Un tipo inactivo no aparece para cotizar pero se conserva en el historial. • Un tipo nuevo está disponible sin redesplegar.
HU-004	Quotation	Must	5	<i>Como administrador, quiero definir reglas básicas de valoración por tipo para que cada equipo se cotee con sus criterios.</i> • Las reglas se guardan versionadas en JSONB. • Cambiar una regla no altera cotizaciones ya emitidas. • Hay al menos una regla activa por tipo.
HU-005	Quotation	Must	8	<i>Como cliente, quiero indicar tipo, marca, modelo, año y condición de mi equipo para recibir un precio estimado.</i> • Solo se listan tipos activos. • La cotización muestra monto y vigencia. • La respuesta tiene p95 menor a 2 s. • Se guardan los datos declarados como snapshot.

Roy

Sprint 2: Aceptación, orden y reglas de negocio (34 pts)

Objetivo: el cliente acepta su cotización y se crea la orden; el proveedor tiene límites de plan.

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación	
TE-02	Infra	Must	3	Mensajería. Broker de eventos y convención de eventos. - Los eventos llevan event_id, correlation_id y tenant_id. - Un consumidor reprocesa un evento duplicado sin efectos dobles.	Roy
HU-007	Catalog	Should	3	<i>Como administrador del catálogo, quiero gestionar marcas por tipo para ofrecer opciones válidas al cotizar.</i> - Crear, editar e inactivar marcas. - Una marca inactiva no se ofrece pero se conserva en las cotizaciones ya hechas.	Fer
HU-008	Catalog	Must	5	<i>Como administrador del catálogo, quiero definir reglas de evaluación (checklist) por tipo para guiar la inspección.</i> - Checklist y criterios flexibles en JSONB. - Reglas versionadas. - La inspección usa la versión vigente.	Fer
HU-009	Quotation	Must	3	<i>Como cliente, quiero consultar mis cotizaciones registradas.</i> - Lista y detalle solo de mis cotizaciones. - Muestra estado y vigencia.	Diego
HU-010	Quotation	Must	5	<i>Como cliente, quiero aceptar o rechazar una cotización.</i> - Solo se acepta una cotización vigente. - Al aceptar se publica QuoteAccepted. - Una cotización vencida pasa a EXPIRED.	Diego
HU-011	Orders	Must	5	<i>Como cliente, quiero que mi aceptación genere una orden para iniciar el reciclaje.</i> - Se crea la orden con snapshot de marca, modelo, año y condición. - La creación es idempotente (un evento repetido no duplica la orden). - Se registra el historial de estados.	Alan
HU-024	Auth/Tenant	Should	5	<i>Como proveedor, quiero un plan gratuito con límites para probar el servicio antes de pagar. (Nueva.)</i> - El plan gratuito permite hasta 20 cotizaciones al mes (\$7). - Al superarlo se bloquea con mensaje claro. - El contador se reinicia cada mes. - El plan premium no tiene límite.	Alan
HU-026	Notification	Should	5	<i>Como cliente, quiero recibir avisos al cambiar el estado de mi solicitud. (Nueva.)</i> - Se envía un correo (simulado o real) al crear la orden y al aceptar la cotización. - Las notificaciones se disparan por eventos, no por llamadas directas. - Los fallos de envío no bloquean el proceso.	Andres

Sprint 3: Logística, inspección y ajuste (32 pts)

Objetivo: el equipo llega, se inspecciona y, si corresponde, se ajusta el precio.

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación
HU-012	Orders	Should	3	<i>Como cliente, quiero solicitar una caja para mi orden.</i> - Se registra la solicitud con dirección. - Una orden admite varias solicitudes. - El estado pasa a BOX_REQUESTED.
HU-013	Orders	Should	5	<i>Como operador de logística, quiero registrar el envío y el tracking.</i>

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación
				- Se registra la guía. - El estado pasa a BOX_SHIPPED e IN_TRANSIT. - El cliente ve el avance.
HU-014	Orders	Must	3	<i>Como operador, quiero registrar la recepción del equipo.</i> - El estado pasa a RECEIVED. - Se publica DeviceReceived. - Catalog actualiza last_received_at del tipo.
HU-015	Inspection	Must	8	<i>Como inspector, quiero registrar la inspección con la checklist del tipo para documentar la condición real.</i> - La checklist corresponde al tipo y a su versión vigente. - Se guardan ítems, observaciones y referencias de evidencia. - Solo un usuario con rol INSPECTOR puede registrar.
HU-016	Inspection	Must	5	<i>Como inspector, quiero determinar el resultado (reventa o reciclaje) para decidir el destino.</i> - El resultado queda registrado con justificación. - Se publica InspectionCompleted. - Orders actualiza la orden a INSPECTED.
HU-017	Quotation	Must	8	<i>Como cliente, quiero recibir una cotización ajustada si la condición real difiere de la declarada.</i> - La ajustada referencia a la original (parent_quote_id) y se justifica por escrito. - Si no hay diferencia, la original se confirma. - El cambio queda auditado.

Sprint 4: Cierre del ciclo, pago y seguimiento (34 pts)

Objetivo: el cliente cobra y el proveedor ve el desempeño del negocio.

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación
HU-018	Quotation	Must	5	<i>Como cliente, quiero aceptar o rechazar el precio ajustado.</i> - Al aceptar se publica AdjustedQuoteAccepted. - Al rechazar, la orden pasa a ADJUSTMENT_REJECTED y se aplica el flujo del supuesto S2.
HU-019	Payment	Must	8	<i>Como cliente, quiero recibir el pago por transferencia o cheque.</i> - Solo se paga con precio final aceptado. - El pago usa clave de idempotencia. - El método queda registrado. - Se guarda el comprobante.
HU-020	Payment	Must	5	<i>Como operador, quiero gestionar reintentos y estados de pago.</i> - Un fallo genera un registro FAILED que se conserva. - Se permite un nuevo intento. - La base impide dos pagos COMPLETED por orden.
HU-025	Orders	Must	5	<i>Como operador, quiero registrar el destino final del equipo (reventa o reciclaje) para cerrar el ciclo. (Nueva.)</i> - Reventa registra la referencia de publicación. - Reciclaje registra un certificado de destrucción. - La orden pasa a DISPOSED y luego a CLOSED.

ID	Módulo	Prioridad	Pts	Historia y criterios de aceptación
HU-021	Notification	Should	3	<i>Como cliente, quiero consultar mis notificaciones y el estado del proceso.</i> - Línea de tiempo de la orden con sus estados. - Lista de avisos enviados.
HU-022	Reporting	Should	5	<i>Como administrador del tenant, quiero ver reportes del negocio.</i> (Redefinida: ya no incluye configuración de tenant.) - Reporte de cotizaciones, órdenes por estado, pagos y destino de equipos. - Se alimenta de eventos, no de lecturas directas a bases ajenas.
HU-023	Catalog	Could	3	<i>Como administrador, quiero que el sistema desactive los tipos sin recepciones en 12 meses.</i> (Nueva.) - Un proceso programado detecta tipos con last_received_at mayor a 12 meses. - Avisa al administrador antes. - Desactiva sin borrar; el historial se conserva.

Resumen de carga

Sprint	Puntos	Contenido
1	34	TE-01, HU-001, 002, 003, 006, 004, 005
2	34	TE-02, HU-007, 008, 009, 010, 011, 024, 026
3	32	HU-012, 013, 014, 015, 016, 017
4	34	HU-018, 019, 020, 025, 021, 022, 023
Total	134	26 HU + 2 habilitadores técnicos

Si el ritmo real baja, se recortan primero HU-023, HU-021, HU-007 y HU-022 (en ese orden): ninguna bloquea el flujo principal.

9. Trazabilidad kata → backlog

Requisito	HU que lo cubren
RF-1 Cotizar	HU-004, 005, 009, 010
RF-2 Caja y envío	HU-011, 012, 013, 014
RF-3 Inspección y destino	HU-015, 016, 025
RF-4 Re-cotización	HU-017, 018
RF-5 Pago	HU-019, 020
RF-6 Catálogo dinámico	HU-006, 007
RF-7 Reglas por tipo	HU-004, 008
RF-8 Retiro anual	HU-023
RF-9 SaaS freemium	HU-001, 024

10. Reglas de trabajo

Definition of Ready. Historia con “como... quiero... para...”, CA claros, dependencias identificadas y estimación acordada.

Definition of Done.

- Código revisado por al menos un compañero.
- Pruebas unitarias y una de integración del servicio.
- Contrato de API o de eventos documentado.
- Desplegado en el entorno de pruebas con docker compose.
- CA verificados por alguien distinto a quien programó.
- Sin FKs entre bases ni consultas directas a bases ajenas.

Ceremonias (1 semana por sprint). Planificación (lunes), daily de 10 minutos, revisión con demo y retrospectiva (viernes).

11. Riesgos

Riesgo	Prob.	Impacto	Mitigación
Sprint 1 sobrecargado (34 pts, base técnica incluida)	Alta	Alto	TE-01 se hace en los dos primeros días; HU-007 y HU-023 absorben el recorte
Pagos duplicados	Media	Alto	Idempotencia y restricción única en PostgreSQL
Fuga de datos entre tenants	Baja	Alto	tenant_id obligatorio en cada consulta; pruebas de aislamiento
Flujo atascado a mitad de proceso	Media	Medio	Máquina de estados definida, timeouts y alertas
Reglas JSONB sin validación	Media	Medio	Esquema de validación por tipo antes de activarlas
Sobrecarga operativa de 6 bases y un broker	Alta	Medio	Un solo PostgreSQL con bases separadas en el MVP
Dependencia de supuestos sin validar	Media	Medio	Confirmar S1 a S8 con el profesor en la primera semana

12. Control de cambios respecto al documento base

Cambio	Motivo
HU-006 pasa del Sprint 2 al 1	Cotizar en el Sprint 1 exige un catálogo de tipos
Auth absorbe Tenant: se agregan tenants, user_roles, tenant_plans, usage_counters	HU-001 hablaba de tenant sin tabla que lo respalde; faltaban roles
HU-022 pasa a ser solo Reporting	Reporting no debe ser dueño de configuración de tenant
Nuevas HU-023, 024, 025, 026	Cubren retiro anual, freemium, destino final y emisión de notificaciones
Orders es dueño del proceso (saga)	El documento base no decía quién coordinaba el flujo
Se agregan TE-01 y TE-02	Infraestructura y mensajería no tenían lugar en el backlog
Se define el camino de rechazo del ajuste	Estaba sin resolver en el kata y en el documento base
Se agregan prioridades MoSCoW, puntos y CA resumidos	Hacían falta para planificar

13. Pendientes de validación

1. Confirmar con el profesor los supuestos S1 a S8, en especial S1, S2 y S7.
2. Confirmar la herramienta de broker (por ejemplo RabbitMQ) y el stack de cada servicio.
3. Definir los contratos REST y de eventos (Sprint 1, durante la planificación).
4. Elaborar los diagramas C4 (contexto, contenedores y componentes) a partir de la sección 4.
5. Redactar ADRs breves para: microservicios con base por servicio, Orders como coordinador, JSONB para reglas, JWT con tenant_id como claim, y adaptadores simulados para banco y marketplace.